

Go

A linguagem que o Google criou enquanto esperava o C++ compilar

Nome 1 · Nome 2

Prof. Bruno Légora

Roteiro

- | | | |
|---|---|---------|
| 1 | Contexto e histórico
os problemas que motivaram Go e quem a criou | ~3 min |
| 2 | Casos de uso
os projetos conhecidos que são escritos em Go | ~3 min |
| 3 | Avaliação de critérios
de aplicabilidade a custo | ~10 min |
| 4 | Avaliações teóricas
escopo, tipos, memória, polimorfismo, concorrência, erros | ~14 min |

PARTE 1

Contexto e histórico

Dois problemas no Google de 2007

COMPILAÇÃO LENTA DE C++

- Programas de milhões de linhas levavam dezenas de minutos para compilar, mesmo num cluster de compilação
- O vilão era o `#include`: cada arquivo reabre e reprocessa os mesmos headers, e o custo cresce com o projeto

CONCORRÊNCIA DIFÍCIL

- Os servidores atendiam milhares de conexões ao mesmo tempo, em máquinas com vários núcleos
- Em C++ e Java isso significava threads do sistema operacional com locks, caras de criar e fáceis de errar

COMO A IDEIA SURTIU

Rob Pike conta que a conversa que deu origem a Go aconteceu em setembro de 2007, durante um build de C++ de uns 45 minutos.

Queriam uma linguagem que compilasse em segundos, que equipes grandes conseguissem ler e que tivesse concorrência na própria sintaxe.

PIKE, "Less is exponentially more", 2012 · PIKE, "Go at Google", 2012

Os três fundadores

Robert Griesemer

Trabalhou na máquina virtual Java HotSpot e no motor JavaScript V8 do Chrome

Rob Pike

Esteve na equipe do Unix e do Plan 9 nos Bell Labs e é coautor do UTF-8

Ken Thompson

Criou o Unix e a linguagem B, que deu origem ao C. Coautor do UTF-8, ganhou o prêmio Turing em 1983

2007

projeto começa no Google

2009

código aberto

2012

Go 1.0 e promessa de compatibilidade

2022

Go 1.18 traz genéricos

hoje

duas versões por ano

PARTE 2

Casos de uso

Se você usa Docker ou Kubernetes, já roda código Go
todo dia

Contêineres e orquestração

- **Docker**, com o `containerd` e o `runc` por baixo, que são os programas que de fato criam os contêineres
- **Kubernetes**, o orquestrador de contêineres que o próprio Google abriu em 2014
- **etcd**, o banco chave-valor onde o Kubernetes guarda o estado do cluster
- **Helm**, o gerenciador de pacotes do Kubernetes
- **Istio** (plano de controle), para redes de serviços

POR QUE GO AQUI?

Um contêiner precisa de um binário que rode sem instalar nada. Go gera um executável estático, que basta copiar para a imagem.

Orquestrar milhares de contêineres é, na prática, manter milhares de conexões de rede abertas. As goroutines foram pensadas para esse tipo de carga.

DevOps, nuvem e observabilidade

INFRAESTRUTURA COMO CÓDIGO (HASHICORP)

- **Terraform**, que cria máquinas e redes na nuvem a partir de arquivos de configuração
- **Vault**, para guardar senhas e chaves, e **Consul**, para descoberta de serviços

MONITORAMENTO

- **Prometheus**, que coleta métricas dos servidores
- **Grafana**, cujo servidor é escrito em Go
- **Jaeger**, para rastrear requisições entre microsserviços

BANCOS E SERVIDORES

- **CockroachDB** e **InfluxDB**, bancos de dados distribuídos
- **Vitess**, criado no YouTube para escalar o MySQL
- **MinIO**, armazenamento de objetos compatível com o S3
- **Caddy** e **Traefik**, servidores web e proxies reversos

Ferramentas do dia a dia e empresas

FERRAMENTAS DE LINHA DE COMANDO

gh (GitHub CLI)

Hugo

Ollama

fzf

lazygit

k9s

Syncthing

Gitea

Uma CLI em Go é distribuída como um único arquivo por sistema operacional. O usuário baixa e roda, sem instalar runtime.

EM PRODUÇÃO

- **Google:** serviços internos e o Viteess do YouTube
- **Uber:** boa parte dos microserviços
- **Cloudflare:** serviços que ficam no caminho do tráfego
- **Dropbox:** migrou do Python para Go os sistemas que precisavam de desempenho
- **Twitch:** o sistema de chat

PARTE 3

Avaliação de critérios

Aplicabilidade

ONDE GO VAI BEM

- É de propósito geral e é mais usada em servidores de rede, APIs, infraestrutura de nuvem e ferramentas de linha de comando (a parte 2 inteira)

ONDE GO VAI MAL

- Interfaces gráficas e aplicativos mobile: a biblioteca padrão não tem nada para isso
- Ciência de dados e aprendizado de máquina, onde o ecossistema é Python
- Sistemas de tempo real e embarcados, porque as pausas do coletor de lixo não são previsíveis

POR QUÊ?

Go foi desenhada para resolver os problemas de um tipo de software, o servidor de rede do Google. Por isso a biblioteca padrão vem com servidor HTTP, criptografia, JSON e testes, e não traz nada de interface gráfica.

Confiabilidade

- Acesso fora dos limites de um array é verificado em tempo de execução e para o programa, em vez de ler lixo de memória como em C
- Não há aritmética de ponteiro
- O coletor de lixo elimina o `free` duplo e o ponteiro solto
- Declarar uma variável e não usar é erro de compilação, assim como importar um pacote sem usar

```
notas := []int{7, 8, 9}
notas[5]
// panic: runtime error: index out of range [5] with length 3
```

Por que só "um pouco" confiável: o ponteiro `nil` continua existindo e derruba o programa, e um erro retornado pode ser ignorado com `_`.

POR QUE NÃO TEM ARITMÉTICA DE PONTEIRO?

"Safety. Without pointer arithmetic it's possible to create a language that can never derive an illegal address that succeeds incorrectly."

Go FAQ

O FAQ acrescenta que um laço com índice já é tão rápido quanto um com ponteiro, e que sem aritmética o coletor de lixo fica mais simples.

POR QUE VARIÁVEL NÃO USADA É ERRO?

"The presence of an unused variable may indicate a bug, while unused imports just slow down compilation."

Go FAQ

Facilidade de aprendizado

- A sintaxe é pequena: 25 palavras reservadas, contra 32 do C e mais de 50 do Java
- Há um jeito de fazer cada coisa: um só laço, um formatador oficial, nenhuma herança
- A concorrência é a cereja do bolo de Go, e para usá-la bem é preciso background de computação: condição de corrida, deadlock, sincronização

```
ch := make(chan int)
ch <- 1          // ninguém vai ler
// fatal error: all goroutines are asleep - deadlock!
```

POR QUE TÃO PEQUENA?

O Google contratava muitos programadores recém-formados, e o código passava por muitas mãos. Pike diz que a linguagem tinha de ser aprendida rápido por quem veio de C, C++ ou Java.

O que dá trabalho em Go são os conceitos. As três linhas ao lado estão sintaticamente certas e travam o programa, e só percebe o motivo quem sabe como um canal sem buffer funciona.

PIKE, "Go at Google", 2012

Eficiência

- Compila para código de máquina nativo, sem máquina virtual e sem interpretador
- Uma goroutine começa com uns poucos KB de pilha, então um programa aguenta centenas de milhares delas
- O coletor de lixo tem um custo em CPU e causa pausas curtas, e é por isso que Go fica atrás de C e Rust em código muito pesado
- Está validada em infraestrutura crítica: Docker e Kubernetes rodam em cima dela

O EQUILÍBRIO ESCOLHIDO

Os autores queriam a velocidade de uma linguagem compilada junto com a comodidade de um coletor de lixo. Aceitaram perder um pouco de desempenho bruto para ganhar compilação rápida e segurança de memória.

Portabilidade

- A biblioteca padrão esconde o que depende do sistema operacional: arquivos, caminhos, rede, tempo
- O mesmo código compila para Linux, Windows e macOS sem `#ifdef`
- O resultado é um único executável, sem dependências
- A compilação cruzada é uma variável de ambiente:

```
GOOS=windows GOARCH=amd64 go build    # gera um .exe no Linux
GOOS=darwin   GOARCH=arm64 go build    # binário para Mac M1
```

POR QUE UM EXECUTÁVEL SÓ?

No Google, colocar um programa em milhares de servidores esbarrava em versões diferentes de bibliotecas instaladas em cada máquina. Com tudo ligado estaticamente no binário, basta copiar um arquivo.

Java resolve a portabilidade com a JVM, que precisa estar instalada. Go resolve recompilando para cada alvo.

Método de projeto

- Principalmente estruturado e imperativo: funções, variáveis, sequência, seleção e repetição
- Tem tipos com métodos e interfaces, mas não tem classes nem herança

```
func media(notas []float64) float64 {  
    soma := 0.0  
    for _, n := range notas {  
        soma += n  
    }  
    return soma / float64(len(notas))  
}
```

POR QUE NÃO TEM HERANÇA?

"Object-oriented programming, at least in the best-known languages, involves too much discussion of the relationships between types, relationships that often could be derived automatically."

Go FAQ

No lugar da hierarquia de classes, Go usa composição (uma struct dentro da outra) e interfaces que o tipo satisfaz sem precisar declarar.

Evolutibilidade

O PROGRAMA EVOLUI COM FACILIDADE

- `gofmt` formata todo código Go do mesmo jeito, então qualquer pessoa lê o código de qualquer outra
- Com tipagem estática, mudar uma assinatura faz o compilador apontar cada lugar que quebrou
- Interfaces pequenas e implícitas deixam trocar uma implementação sem mexer em quem a usa

A LINGUAGEM EVOLUI SEM QUEBRAR

- Desde o Go 1.0 (2012), o código continua compilando nas versões novas
- Os genéricos entraram na 1.18 sem invalidar nada antigo
- Quando um comportamento muda, ele só vale para quem declara a versão nova no `go.mod`
- `go fix` reescreve automaticamente o uso de APIs antigas

A PROMESSA DE COMPATIBILIDADE

"Programs written to the Go 1 specification will continue to compile and run correctly, unchanged, over the lifetime of that specification."

Go 1 and the Future of Go Programs

"Gofmt's style is no one's favorite, yet gofmt is everyone's favorite."

Go Proverbs, Rob Pike

Python, por comparação, quebrou boa parte do código existente na passagem da versão 2 para a 3.

Reusabilidade

- Permite polimorfismo paramétrico: uma função ou tipo genérico serve para vários tipos (detalhes na parte 4)
- Interfaces fazem uma função aceitar qualquer tipo que tenha os métodos pedidos
- O princípio DIY (*do it yourself*) é muito forte na comunidade: em vez de importar uma biblioteca pequena, escreve-se o próprio código com a biblioteca padrão

```
func Maior[T cmp.Ordered](a, b T) T {  
    if a > b {  
        return a  
    }  
    return b  
}  
Maior(3, 9)           // 9  
Maior("go", "ada")    // "go"
```

POR QUE A CULTURA DIY?

"A little copying is better than a little dependency."

Go Proverbs, Rob Pike

Cada dependência traz código que você não controla, atualizações que podem quebrar e mais tempo de compilação. A biblioteca padrão já cobre HTTP, JSON, criptografia e testes, então dá para fazer muita coisa sem biblioteca externa.

A contrapartida é que a comunidade reutiliza menos do que a linguagem permite.

Integração com outros softwares

- O `cgo` integra Go com C: o código C vai num comentário logo acima de `import "C"`
- Assim dá para usar bibliotecas C que já existem, como SQLite e OpenSSL
- Go também pode gerar uma biblioteca compartilhada que outro programa em C chama

```
/*  
#include <math.h>  
#cgo LDFLAGS: -lm  
*/  
import "C"  
  
fmt.Println(C.sqrt(2))    // 1.4142135623730951
```

POR QUE A COMUNIDADE EVITA O CGO?

"Cgo is not Go."

Go Proverbs, Rob Pike

Chamar C custa caro, porque a goroutine precisa trocar para uma pilha de sistema. Com cgo o binário deixa de ser independente e a compilação cruzada complica.

Por isso muitos projetos reescrevem a biblioteca em Go puro, mais um caso da cultura DIY.

Custo

- Linguagem open source desde 2009, com licença BSD
- Compilador, gerenciador de dependências, formatador, testes e análise estática vêm juntos e de graça
- Funciona em qualquer editor

OS OUTROS CUSTOS

Treinar alguém em Go é barato porque a linguagem é pequena. Executar também, porque o código é nativo e usa pouca memória.

A manutenção é barata pela formatação única e pela promessa de compatibilidade. Onde Go custa mais é na verbosidade: o tratamento de erro repete muitas linhas.

PARTE 4

Avaliações teóricas

Os conceitos vistos em aula, aplicados a Go

Escopo

- O escopo é estático (léxico): dá para saber a qual declaração um nome se refere lendo o texto do programa, sem executar
- Cada par de chaves abre um bloco, e um nome declarado dentro oculta o de fora
- Os níveis são universo, pacote, arquivo, função e bloco

```
var x = "pacote"

func main() {
    fmt.Println(x)      // pacote
    x := "função"
    {
        x := "bloco"
        fmt.Println(x)  // bloco
    }
    fmt.Println(x)      // função
}
```

POR QUE ESTÁTICO?

O compilador resolve cada nome antes de rodar o programa, então não há busca de variáveis em tempo de execução e quem lê o código sabe de onde cada nome vem. É o mesmo modelo de C, Java e da maioria das linguagens atuais. Escopo dinâmico ficou restrito a casos como o Lisp antigo e variáveis de shell.

Expressões e comandos: sem ternário

- Go não tem o operador `?:`
- O `if` é um comando e não uma expressão: ele não devolve valor, então não dá para escrever `x := if ...`
- Atribuição e `i++` também são comandos, então `if (x = 5)` e `y = i++` não compilam

```
maior := a > b ? a : b
// invalid character U+003F '?'
```

```
maior := if a > b { a } else { b }
// syntax error: unexpected keyword if, expected expression
```

```
maior := a
if b > a {
    maior = b
}
```

POR QUE NÃO TEM TERNÁRIO?

"The reason `?:` is absent from Go is that the language's designers had seen the operation used too often to create impenetrably complex expressions. The if-else form, although longer, is unquestionably clearer. A language needs only one conditional control flow construct."

Go FAQ

Pelo mesmo raciocínio, `if` não virou expressão: seria uma segunda forma de condicional. E tirar a atribuição das expressões elimina o clássico `if (x = 5)` do C, que compila e está errado.

Expressões e comandos: sem while

```
for i := 0; i < 10; i++ { }    // for clássico de C

for n > 0 {                  // o "while"
    n--
}

for {                          // laço infinito
    if pronto() { break }
}

for i, nota := range notas { } // percorre coleções
```

Um único comando de repetição cobre todos os casos. Também não existe `do-while`.

POR QUE SÓ `for` ?

Um `while` é um `for` sem as cláusulas de início e incremento. Ter duas palavras para a mesma coisa não acrescenta poder à linguagem, só mais uma regra para aprender.

"It unifies for and while and there is no do-while."

Effective Go

Vale o mesmo critério do ternário, uma construção por conceito. É assim que a linguagem fica com 25 palavras reservadas.

Tipos primitivos

- Não tem `char`. Um caractere é uma runa (`rune`, apelido de `int32`), que guarda um ponto de código Unicode
- `byte` é apelido de `uint8`, e `string` é uma sequência imutável de bytes em UTF-8
- Não tem `enum`. Usa-se constantes com `iota`, que conta 0, 1, 2...
- Os demais: `bool`, inteiros de 8 a 64 bits com e sem sinal, `float32`, `float64` e complexos

```
len("çãõ")           // 5 (bytes)
len([]rune("çãõ"))   // 3 (runas)
```

```
type Dia int
const (
    Domingo Dia = iota // 0
    Segunda           // 1
    Terca              // 2
)
```

POR QUE RUNA E NÃO CHAR?

Em C, `char` é um byte, e um byte não comporta "ç" nem "ã" em UTF-8. Pike e Thompson criaram o UTF-8, e fizeram a linguagem separar as duas ideias: byte é byte, e o caractere Unicode ganhou nome próprio.

POR QUE NÃO TEM ENUM?

O FAQ não dá um motivo. Na prática, `iota` resolve o caso comum sem uma construção nova. O custo é que `Segunda + Terca` compila, coisa que um enum de verdade proibiria.

Tipos compostos

```
// array: tamanho fixo e faz parte do tipo
notas := [3]float64{7, 8.5, 9}

// slice: uma janela sobre um array, que cresce
nomes := []string{"ana"}
nomes = append(nomes, "caio")

// map: tabela hash
idade := map[string]int{"ana": 20}

// struct: produto cartesiano
type Aluno struct {
    Nome    string
    Notas []float64
}
```

POR QUE ARRAY E SLICE?

O array é um valor de tamanho fixo e é copiado inteiro na atribuição. É previsível, mas pouco prático. O slice é uma pequena estrutura (ponteiro, tamanho e capacidade) que aponta para um array e cresce com `append`.

Em Go, quase todo código usa slices. O array fica por baixo, como o armazenamento.

POR QUE MAP EMBUTIDO?

Tabela hash é tão comum em servidores que os autores a puseram na linguagem, com sintaxe própria, em vez de numa biblioteca.

Interfaces

```
type Forma interface {  
    Area() float64  
}  
  
type Circulo struct{ Raio float64 }  
  
func (c Circulo) Area() float64 {  
    return math.Pi * c.Raio * c.Raio  
}  
  
// Circulo é uma Forma sem declarar nada  
  
formas := []Forma{Circulo{1}, Quadrado{2}}  
for _, f := range formas {  
    fmt.Println(f.Area())  
}
```

POR QUE NÃO TEM `implements` ?

"A Go type implements an interface by implementing that interface's methods, nothing more. This property allows interfaces to be defined and used without needing to modify existing code."

Go FAQ

Dá para criar uma interface depois que o tipo já existe, até para tipos de uma biblioteca que você não pode editar.

"The bigger the interface, the weaker the abstraction."

Go Proverbs

Sistema de tipos

CLASSIFICAÇÃO	GO	COMO APARECE
Estática ou dinâmica	Estática	todo tipo é conhecido na compilação
Forte ou fraca	Forte	nenhuma conversão implícita, nem de <code>int</code> para <code>float64</code>
Manifesta ou inferida	As duas	<code>var x float64 = 1</code> ou <code>x := 1.0</code>
Compatibilidade	Nominal para tipos com nome, estrutural para interfaces	ver abaixo

```
type Celsius float64
type Fahrenheit float64

var c Celsius = 30
var f Fahrenheit = c // erro: tipos diferentes
f = Fahrenheit(c*9/5 + 32) // conversão explícita
```

POR QUE OS DOIS TIPOS DE COMPATIBILIDADE?

Com compatibilidade nominal, o compilador impede somar `Celsius` com `Fahrenheit`, mesmo sendo os dois `float64` por baixo.

Nas interfaces a compatibilidade é estrutural, então qualquer tipo com os métodos certos serve, inclusive tipos escritos antes da interface existir.

Polimorfismo

- Não tem sobrecarga: duas funções não podem ter o mesmo nome com parâmetros diferentes, e operadores não podem ser redefinidos
- Tem polimorfismo paramétrico desde a 1.18: funções e tipos recebem parâmetros de tipo, limitados por uma restrição
- Tem polimorfismo de inclusão pelas interfaces, com despacho dinâmico
- Não tem coerção, porque não há conversão implícita

```
type Numero interface {  
    ~int | ~int64 | ~float64  
}  
  
func Soma[T Numero](xs []T) T {  
    var total T  
    for _, x := range xs { total += x }  
    return total  
}  
  
Soma([]int{1, 2, 3})           // 6  
Soma([]float64{1.5, 2.5})     // 4  
  
type Pilha[T any] struct{ itens []T }
```

POR QUE NÃO TEM SOBRECARGA?

"Experience with other languages told us that having a variety of methods with the same name but different signatures was occasionally useful but that it could also be confusing and fragile in practice."

Go FAQ

POR QUE OS GENÉRICOS DEMORARAM 13 ANOS?

Durante anos o FAQ dizia que genéricos custam complexidade no sistema de tipos e no runtime, e que a equipe ainda não tinha achado um desenho que valesse esse custo. Várias propostas foram descartadas até a 1.18, em 2022.

Encapsulamento e proteção

- A unidade de encapsulamento é o pacote e não a classe
- Só existem dois níveis de visibilidade, e quem decide é a primeira letra do nome: maiúscula é pública fora do pacote, minúscula é privada ao pacote
- Não existe `protected` nem `friend`
- Um pacote dentro de uma pasta `internal/` só pode ser importado pelo próprio projeto

```
package conta

type Conta struct {
    Titular string // público
    saldo   float64 // privado ao pacote
}

func (c *Conta) Depositar(v float64) {
    c.saldo += v
}
```

POR QUE A LETRA MAIÚSCULA?

Em Java, para saber se um membro é público, você precisa ir até a declaração. Em Go basta olhar o nome no ponto em que é usado: `conta.Depositar` é público, `c.saldo` não.

Encapsular por pacote em vez de por classe combina com a falta de classes: tipos do mesmo pacote enxergam os campos privados uns dos outros, como partes de um mesmo módulo.

Gerenciamento de memória

- Tem coletor de lixo concorrente, que roda junto com o programa. Não existe `malloc` nem `free`
- O compilador decide se uma variável fica na pilha ou no monte olhando o tempo de vida dela, num processo chamado análise de escape
- Se a variável continua sendo usada depois que a função termina, vai para o monte. Se não, fica na pilha

```
func novoContador() *int {  
    n := 0  
    return &n // em C isto é um bug;  
}             // em Go, n vai para o monte
```

```
$ go build -gcflags=-m  
./main.go:2:2: moved to heap: n
```

POR QUE COLETOR DE LIXO?

Para o FAQ, controlar o tempo de vida de cada objeto é uma das maiores fontes de trabalho em programas de sistema. Com concorrência fica pior, porque é difícil saber quando nenhuma goroutine usa mais um dado.

POR QUE O PROGRAMADOR NÃO ESCOLHE PILHA OU MONTE?

"From a correctness standpoint, you don't need to know."

Go FAQ

Em qualquer dos dois lugares o programa funciona igual. A escolha só muda o desempenho, então o compilador decide sozinho.

Persistência

- A linguagem não tem persistência embutida: ela vem da biblioteca padrão
- Duas interfaces abstraem toda entrada e saída: `io.Reader` e `io.Writer`
- Arquivo, conexão de rede, terminal, buffer de memória e compressão implementam as duas
- Por cima delas ficam `encoding/json`, `encoding/csv`, `encoding/gob` e `database/sql`

```
type Reader interface {  
    Read(p []byte) (n int, err error)  
}  
  
type Writer interface {  
    Write(p []byte) (n int, err error)  
}  
  
func salvar(w io.Writer, a Aluno) error {  
    return json.NewEncoder(w).Encode(a)  
}  
  
salvar(arquivo, ana)    // grava no disco  
salvar(os.Stdout, ana)  // mostra no terminal  
salvar(conexao, ana)    // manda pela rede
```

POR QUE INTERFACES TÃO PEQUENAS?

Um método só é fácil de implementar, então quase tudo acaba sendo um `Reader` ou um `Writer`. Assim a mesma função `salvar` serve para disco, terminal e rede sem mudar uma linha.

Também dá para empilhar: um `gzip.Writer` recebe outro `Writer` e comprime o que passa por ele. É a ideia dos pipes do Unix, que Pike e Thompson ajudaram a criar.

Passagem de parâmetros

- Só existe passagem por cópia
- Para alterar o original, passa-se um ponteiro, e o próprio ponteiro é copiado
- Slices e maps parecem passagem por referência porque a cópia contém um ponteiro para os mesmos dados

```
func dobrar(x int)      { x *= 2 }  
func dobrarP(x *int)   { *x *= 2 }
```

```
n := 5  
dobrar(n)    // n continua 5  
dobrarP(&n)  // n vira 10
```

```
func zerar(s []int) { s[0] = 0 }  
s := []int{1, 2, 3}  
zerar(s)           // s vira [0 2 3]
```

POR QUE SÓ POR CÓPIA?

"As in all languages in the C family, everything in Go is passed by value."

Go FAQ

Em C++, `f(x)` pode alterar `x` se o parâmetro for uma referência, e quem lê a chamada não percebe. Em Go, se a função pode alterar a variável, a chamada mostra isso com `&n`.

Concorrência: goroutines

- Uma goroutine é uma função executando de forma concorrente. Para criar uma, basta escrever `go` antes da chamada
- Quem gerencia as goroutines é o runtime de Go e não o sistema operacional. O escalonador distribui milhares delas sobre poucas threads (modelo M:N)
- Cada uma começa com uma pilha de poucos KB que cresce conforme precisa. Uma thread do sistema reserva 8 MB no Linux

```
func atender(conn net.Conn) {  
    defer conn.Close()  
    // lê e responde a um cliente  
}  
  
for {  
    conn, _ := servidor.Accept()  
    go atender(conn)    // uma goroutine por cliente  
}
```

POR QUE NÃO USAR THREADS DO SISTEMA?

Threads são caras de criar e de trocar, então um servidor em C++ ou Java não cria uma para cada cliente. Ele recorre a pools de threads ou a callbacks, e o código fica difícil de seguir.

Com goroutines baratas, cada cliente ganha a sua e o código de `atender` é sequencial e simples. Quem cuida de encaixar tudo nos núcleos é o runtime.

Concorrência: canais

- Um canal é um tubo tipado por onde goroutines trocam valores. A comunicação entre threads faz parte da linguagem: `chan`, `<-` e `select` são sintaxe
- Sem buffer, quem envia espera quem recebe, e os dois se encontram (rendezvous). Com buffer, o canal é uma fila limitada
- O tipo pode restringir a direção: `chan<- int` só envia e `<-chan int` só recebe
- `close` avisa que acabou, e o `range` do outro lado termina sozinho
- `select` espera em vários canais e segue pelo primeiro que ficar pronto

```
func produtor(saida chan<- int) {
    for i := 0; i < 5; i++ {
        saida <- i           // envia
    }
    close(saida)
}

fila := make(chan int, 2) // buffer de 2
go produtor(fila)
for v := range fila {     // recebe até fechar
    fmt.Println(v)
}

select {
case msg := <-entrada:
    tratar(msg)
case <-time.After(time.Second):
    fmt.Println("timeout")
}
```

Concorrência: por que canais?

A BIBLIOTECA PADRÃO TAMBÉM TEM O MODELO CLÁSSICO

- `sync.Mutex` e `sync.RWMutex` para exclusão mútua
- `sync.WaitGroup` para esperar um grupo de goroutines terminar
- `sync.Once` para inicializar algo uma vez só, e `sync/atomic` para operações atômicas
- `go test -race` detecta condições de corrida enquanto os testes rodam

```
var mu sync.Mutex
mu.Lock()
contador++
mu.Unlock()
```

A FILOSOFIA

"Don't communicate by sharing memory, share memory by communicating."

Go Proverbs, Rob Pike

O modelo vem do CSP, *Communicating Sequential Processes*, de Tony Hoare (1978). Em vez de várias threads mexendo no mesmo dado sob um lock, um dado pertence a uma goroutine de cada vez e passa para a próxima pelo canal.

O mutex continua ali para casos simples, como um contador. Para coordenar trabalho, a linguagem empurra para os canais.

Tratamento de erros

- Go não tem exceções. Um erro é um valor comum do tipo `error`, devolvido como último retorno da função
- Quem chama verifica na hora, com `if err != nil`
- Um erro pode embrulhar outro com `%w`, e `errors.Is` procura um erro específico na cadeia
- `panic` existe só para falhas de programação, como índice inválido

```
f, err := os.Open("notas.txt")
if err != nil {
    return fmt.Errorf("abrindo notas: %w", err)
}
defer f.Close()
```

POR QUE NÃO TEM EXCEÇÃO?

"We believe that coupling exceptions to a control structure, as in the try-catch-finally idiom, results in convoluted code. It also tends to encourage programmers to label too many ordinary errors, such as failing to open a file, as exceptional."

Go FAQ

Falhar ao abrir um arquivo é algo esperado, então vira um valor tratado como qualquer outro, e dá para ver na própria função todos os caminhos que o programa pode seguir.

O preço é a verbosidade. Propostas para encurtar o `if err != nil` já foram feitas e rejeitadas pela comunidade.

Conclusão

- Go nasceu de dois incômodos do Google: compilação lenta e concorrência difícil
- Quase toda ausência é uma escolha: sem ternário, sem while, sem herança, sem sobrecarga, sem exceção. Os autores preferiram uma construção por conceito
- O que ela traz de diferente é a concorrência dentro da linguagem, com goroutines e canais
- Por isso domina a infraestrutura de nuvem, de Docker a Kubernetes, e fica de fora de GUI, ciência de dados e tempo real

Referências

- The Go Programming Language Specification. go.dev/ref/spec
- Frequently Asked Questions (FAQ). go.dev/doc/faq
- Effective Go. go.dev/doc/effective_go
- Go 1 and the Future of Go Programs. go.dev/doc/go1compat
- PIKE, R. *Go at Google: Language Design in the Service of Software Engineering*. SPLASH, 2012. go.dev/talks/2012/splash.article
- PIKE, R. *Less is exponentially more*. 2012. commandcenter.blogspot.com
- PIKE, R. *Go Proverbs*. Gopherfest, 2015. go-proverbs.github.io
- HOARE, C. A. R. *Communicating Sequential Processes*. CACM, v. 21, n. 8, 1978.
- DONOVAN, A.; KERNIGHAN, B. *The Go Programming Language*. Addison-Wesley, 2015.
- VAREJÃO, F. *Linguagens de Programação: Conceitos e Técnicas*. Campus, 2004.

Obrigado



Perguntas?

Nome 1 · Nome 2